Customizing the vehicle14 outer model and input signals

Detailed practical guide for creating custom simulations without editing the generated dynamics file.

Scope of this PDF

This guide explains how to customize the user-editable outer model in the current MATLAB/CasADi package. It focuses on the file that defines driver signals, total drive torque, total brake torque, steering, torque split, tyre/contact forces, and aero inputs. It also explains how those quantities are converted into the 34-entry input vector used by the generated dynamics file.

The generated dynamics file should be treated as a black box: it returns a mass matrix M, a forcing vector F, and a sensor vector S. You normally do not edit it by hand. Custom simulations should instead be created by editing the outer model file and, when needed, the runner options.

Terminology note: this document deliberately uses neutral names such as generated dynamics file, generated model, and outer model.

1. Big picture: what you are customizing

The simulation has two layers. The generated dynamics layer computes the mechanical equations and wheel geometry sensors. The outer model layer computes everything that comes from outside the mechanical skeleton: steering, tyre/contact forces, wheel torques, braking, powertrain, and aero. This separation is useful because you can change the vehicle control and tyre model without regenerating the equations.

Step	What happens	Where to customize
1	You run a MATLAB command that chooses simulate/plots/render, outer/original input source, and flat/ribbon road.	Run command arguments.
2	The runner builds parameters, options, road, and the generated M/F/S function.	Runner options if you need duration, tolerances, road shape, or filenames.
3	For outer simulations, the outer model is called in control mode to return only steering angle and steering rate.	vehicle14_outer_models_user -> user_control_signals and steering_only.
4	The generated function evaluates sensor vector S using that steering. S contains wheel positions, wheel velocities, wheel axes, and carrier angular velocities.	Usually do not edit. Use S as input to your tyre/contact model.
5	The outer model is called in full mode. It computes full input vector IN: tyre/contact wrenches, wheel torques, and aero.	vehicle14_outer_models_user.
6	The runner calls MATLAB ode15i with residual M*xdot - F = 0. If that fails, it uses ode15s with xdot = M\F.	Only edit if you want different solver tolerances or solver policy.

2. The three files you should understand

File role	What it does	Should you edit it?
Main runner	Parses run arguments, creates road and options, initializes the state, calls MATLAB solvers, saves plots and animations.	Sometimes. Edit for solver settings, road options, output names, or plotting.
Generated dynamics file	Defines the generated M, F, and S formulas. It expects q, u, parameter vector P, and input vector IN.	Normally no. Regenerate rather than hand-edit.
Outer model file	Defines user controls, powertrain/brake split, tyre/contact model, aero model, and fills IN(34).	Yes. This is the main customization file.

For day-to-day custom simulations, most changes should go into the outer model file only.

3. How to choose outer/original and flat/ribbon road

The current runner accepts flexible arguments. The default is the user-editable outer model with the 3D ribbon road. You can explicitly choose the input source and road type from the MATLAB command line.

```
% Default: outer model + ribbon road
run_vehicle14_matlab_casadi_demo
```

```
% Outer model + flat road
run_vehicle14_matlab_casadi_demo('simulate','outer','flat')

% Outer model + 3D ribbon road
run_vehicle14_matlab_casadi_demo('simulate','outer','ribbon')

% Built-in component model + flat road
run_vehicle14_matlab_casadi_demo('simulate','original','flat')

% Built-in component model + 3D ribbon road
run_vehicle14_matlab_casadi_demo('simulate','original','ribbon')
```

Use the flat road first when debugging a new control law or tyre model. The flat road removes elevation and banking so sign errors are easier to detect. After the model behaves correctly on the flat road, run the same setup on the ribbon road.

4. The input vector IN(34): what the outer model must fill

The generated dynamics function receives a 34-entry input vector. The outer model constructs this vector. The first two entries are steering and steering rate. Entries 3 to 26 are body-frame wheel-centre force/moment components. Entries 27 to 30 are wheel spin torques. Entries 31 to 34 are front/rear aero drag and downforce inputs.

IN index range	Meaning	Filled by
1	Steering angle delta.	user_control_signals -> IN(meta.input_index.delta)
2	Steering rate delta_dot.	user_control_signals / steering_only -> IN(meta.input_index.delta_dot)
3-8	FL body-frame tyre/contact force and moment: FBx,FBy,FBz,MBx,MBy,MBz.	user_tyre_contact_wrench
9-14	FR body-frame tyre/contact force and moment.	user_tyre_contact_wrench
15-20	RL body-frame tyre/contact force and moment.	user_tyre_contact_wrench
21-26	RR body-frame tyre/contact force and moment.	user_tyre_contact_wrench
27-30	Wheel spin torques TFL,TFR,TRL,TRR.	user_powertrain_brake_model
31-34	Aero drag/downforce inputs FdragF,FdownF,FdragR,FdownR.	user_aero_model

5. The two-stage outer-model call

The outer model is intentionally called twice during each RHS/residual evaluation. This is necessary because the wheel sensor geometry depends on steering.

Mode	Inputs available	Output produced	Why it exists
control	t, x, parameters, options, road, metadata. S is empty.	IN with only delta and delta_dot plus control diagnostics.	The generated sensor geometry needs the current steering angle.
full	t, x, sensor vector S, parameters, options, road, metadata.	Complete IN(34), including tyre/contact forces, wheel torques, and aero.	The tyre model needs wheel positions, velocities, and wheel axes from S.

```
% Conceptual data path inside the runner for the outer model:
[IN0, ctrl] = vehicle14_outer_models_user('control', t, x, [], par, args, road, meta);
[~,~,Sdm]   = mf_fun(q, u, par.P, IN0);
S           = full(Sdm);
[IN, outer] = vehicle14_outer_models_user('full', t, x, S, par, args, road, meta);
[Mdm,Fdm,~] = mf_fun(q, u, par.P, IN);
```

Important rule: any steering signal you define in user_control_signals must be matched by the steering_only helper, because steering_only is used to estimate delta_dot by finite difference. If the two differ, the steering angle and steering rate will be inconsistent.

6. Where to customize input signals

The main control function is user_control_signals. It creates normalized driver commands named throttle and brake, converts them into total torques, and returns steering angle and steering rate.

Variable	Meaning	Default use
throttle	Dimensionless command, normally 0 to 1.	ctrl.T_drive_total = cfg.T_drive_max * throttle
brake	Dimensionless command, normally 0 to 1.	ctrl.T_brake_total = cfg.T_brake_max * brake

Variable	Meaning	Default use
delta	Steering angle in radians.	Sent to IN(1) and used by front wheel geometry.
delta_dot	Steering rate in rad/s.	Sent to IN(2) and used by velocity-level steering geometry.
T_drive_total	Total positive driving torque in N m.	Split to rear wheels in user_powertrain_brake_model.
T_brake_total	Total positive brake torque magnitude in N m.	Split to four wheels with front bias and optional left/right redistribution.

6.1 Quick customization by changing cfg values

For simple tests, you can keep the existing control profiles and only change numerical values in user_outer_model_options. This is the safest first step because it does not change function structure.

```
% In user_outer_model_options:
cfg.throttle_level = 0.30;     % lower acceleration command
cfg.steer_deg      = 3.0;      % smaller steering amplitude
cfg.brake_start    = 8.0;      % braking begins earlier after settle time
cfg.brake_level    = 0.15;     % gentler braking
cfg.tyre_model     = 'linear-saturated';  % simpler tyre force model
```

6.2 Add a new control profile

To add a new maneuver, add another elseif branch inside user_control_signals and add the same steering logic inside steering_only. The example below creates a simple flat-road launch with a smooth steering pulse.

```
% In user_control_signals, add this branch:
elseif strcmp(profile, 'my_launch_pulse')
    throttle = 0.45 * smooth_step(tau, 0.2, 1.0);
    brake = 0.0;

    pulse_on  = smooth_step(tau, 3.0, 3.5);
    pulse_off = smooth_step(tau, 5.0, 5.5);
    delta = deg2rad(4.0) * pulse_on * (1.0 - pulse_off);

% In steering_only, add the corresponding steering branch:
elseif strcmp(profile, 'my_launch_pulse')
    pulse_on  = smooth_step(tau, 3.0, 3.5);
    pulse_off = smooth_step(tau, 5.0, 5.5);
    delta = deg2rad(4.0) * pulse_on * (1.0 - pulse_off);
```

Then select the profile by setting cfg.control_profile to the new string, or by setting args.maneuver_profile before cfg reads it.

```
% In user_outer_model_options:
cfg.control_profile = 'my_launch_pulse';
```

6.3 Use direct torque commands instead of throttle/brake scaling

The existing function computes total torque as cfg.T_drive_max times throttle and cfg.T_brake_max times brake. If you want to prescribe actual torque directly, you can bypass throttle and brake at the end of user_control_signals.

```
% Replace the final assignment block in user_control_signals with something like:
ctrl.throttle = throttle;
ctrl.brake = brake;
ctrl.T_drive_total = 800.0 * smooth_step(tau, 0.5, 1.5);  % N m, total drive
ctrl.T_brake_total = 0.0;                                  % N m, positive magnitude
ctrl.delta = delta;
ctrl.delta_dot = delta_dot;
```

Keep braking torque magnitude positive in ctrl.T_brake_total. The powertrain/brake split function applies the negative sign when writing wheel brake torques.

6.4 Use a measured or precomputed actuator trace

A common workflow is to define time arrays for drive torque, brake torque, and steering, then interpolate them. Use t_abs for absolute simulation time. If the vehicle first drops and settles, you may prefer tau = t_abs - cfg.drop_settle_time for maneuver time.

```
% Example inside user_control_signals after tau is defined:
tab_t     = [0; 1; 3; 6; 9; 12];
tab_Tt    = [0; 300; 900; 900; 0; 0];       % N m
tab_Tb    = [0; 0; 0; 0; 1200; 0];          % N m, positive magnitude
tab_delta = deg2rad([0; 0; 3; -3; 0; 0]);   % rad

Tt_direct = interp1(tab_t, tab_Tt, tau, 'linear', 'extrap');
Tb_direct = interp1(tab_t, tab_Tb, tau, 'linear', 'extrap');
delta     = interp1(tab_t, tab_delta, tau, 'pchip', 'extrap');
```

```
% Then assign these directly:
ctrl.T_drive_total = max(Tt_direct, 0.0);
ctrl.T_brake_total = max(Tb_direct, 0.0);
ctrl.delta = delta;
```

If you use interpolation for steering, update steering_only to use the same interpolation. Otherwise delta_dot will not match delta.

7. How drive and brake torques are converted to wheel torques

The function user_powertrain_brake_model takes total driving torque and total braking torque and maps them to four wheel torques. Positive wheel torque drives the wheel spin forward. Negative wheel torque brakes the wheel.

Computation	Meaning
Tt = ctrl.T_drive_total	Total positive driving torque magnitude.
Tb = ctrl.T_brake_total	Total positive brake torque magnitude.
Tbf = kb*Tb	Front axle brake torque from brake bias.
Tbr = (1-kb)*Tb	Rear axle brake torque from brake bias.
Tdrive_RL, Tdrive_RR	Rear drive torque split, with optional smooth differential bias.
wheel_torque.FL/FR	Front wheel torques are braking only in the current rear-drive setup.
wheel_torque.RL/RR	Rear wheel torques combine drive torque and brake torque.

```
% Current sign convention in user_powertrain_brake_model:
wheel_torque.FL = -(0.5 - zf)*Tbf;
wheel_torque.FR = -(0.5 + zf)*Tbf;
wheel_torque.RL = Tdrive_RL - (0.5 - zr)*Tbr;
wheel_torque.RR = Tdrive_RR - (0.5 + zr)*Tbr;
```

The optional zf and zr terms redistribute brake torque left-to-right using a lateral acceleration proxy ay_proxy = vx*r. They are clamped so that brake splits remain physically reasonable. Disable this redistribution by setting cfg.enable_lateral_brake_redistribution = false.

7.1 Change to equal braking only

```
% In user_outer_model_options:
cfg.enable_lateral_brake_redistribution = false;

% Braking then becomes approximately:
% FL = FR = -0.5*kb*Tb
% RL = drive_RL - 0.5*(1-kb)*Tb
% RR = drive_RR - 0.5*(1-kb)*Tb
```

7.2 Change to four-wheel drive

To test four-wheel drive, modify user_powertrain_brake_model so the drive torque is distributed to all wheels. The example below uses a front-drive fraction gammaF.

```
gammaF = 0.35;  % 35 percent of drive torque to the front axle
Tdrive_FL = 0.5 * gammaF * Tt;
Tdrive_FR = 0.5 * gammaF * Tt;
Tdrive_RL = 0.5 * (1.0 - gammaF) * Tt - diff_bias;
Tdrive_RR = 0.5 * (1.0 - gammaF) * Tt + diff_bias;

wheel_torque.FL = Tdrive_FL - (0.5 - zf)*Tbf;
wheel_torque.FR = Tdrive_FR - (0.5 + zf)*Tbf;
wheel_torque.RL = Tdrive_RL - (0.5 - zr)*Tbr;
wheel_torque.RR = Tdrive_RR - (0.5 + zr)*Tbr;
```

8. How the tyre/contact model receives its inputs

The tyre/contact model should use the sensor vector S, not manually reconstructed wheel geometry. In the current outer model, each wheel loop extracts wheel centre position, wheel centre velocity, wheel heading axis, wheel spin axis, carrier angular velocity, and wheel angular speed.

Quantity	Source in code	Used for
p_wc_N	S(_Wc_x/y/z)	Wheel centre position in global frame.
v_wc_N	S(_Wc_vx/vy/vz)	Wheel centre velocity in global frame.
head_N	S(_head_Nx/Ny/Nz)	Wheel forward/heading direction.
spin_N	S(_spin_Nx/Ny/Nz)	Wheel spin axis direction.

Quantity	Source in code	Used for
carrier_omega_N	S(_carrier_wx/wy/wz)	Angular velocity of the non-spinning wheel carrier.
omega	u(10+ii)	Wheel spin rate for FL/FR/RL/RR.

```
% Current per-wheel extraction pattern:
p_wc_N = [S(idx.([nm '_Wc_x'])); S(idx.([nm '_Wc_y'])); S(idx.([nm '_Wc_z']))];
v_wc_N = [S(idx.([nm '_Wc_vx'])); S(idx.([nm '_Wc_vy'])); S(idx.([nm '_Wc_vz']))];
head_N = [S(idx.([nm '_head_Nx'])); S(idx.([nm '_head_Ny'])); S(idx.([nm '_head_Nz']))];
spin_N = [S(idx.([nm '_spin_Nx'])); S(idx.([nm '_spin_Ny'])); S(idx.([nm '_spin_Nz']))];
omega = uv(10+ii);
```

This is the correct place to connect any tyre model, because the wheel geometry already includes suspension travel and steering.

9. Contact geometry and slip calculation

The function contact_kinematics_from_pose computes contact-point velocities and compression rates. The function user_tyre_contact_wrench then computes normal force, longitudinal slip, lateral slip, and tyre forces.

Code variable	Meaning
compression	Positive means the tyre is compressed against the road surface.
compression_rate	Rate of change of road-normal compression. Used in vertical tyre damping.
Fz	Normal force computed with a smooth positive function: smoothplus(kt*compression + ct*compression_rate).
Vx	Carrier contact velocity projected along road longitudinal tangent.
Vy	Carrier contact velocity projected along road lateral tangent.
rolling_speed	Wheel rolling speed from spin velocity projected along road longitudinal tangent.
kappa	Longitudinal slip: (rolling_speed - Vx)/sqrt(Vx^2 + v_eps^2).
alpha	Lateral slip angle: atan2(Vy, sqrt(Vx^2 + v_eps^2)).

```
Fz = smoothplus(cfg.kt*compression + cfg.ct*compression_rate, cfg.normal_smooth_eps);
Vx = dot(v_contact_carrier_N, t_long_N);
Vy = dot(v_contact_carrier_N, t_lat_N);
rolling_speed = -dot(v_spin_contact_N, t_long_N);
Vden = sqrt(Vx*Vx + cfg.v_eps*cfg.v_eps);
kappa = (rolling_speed - Vx)/Vden;
alpha = atan2(Vy, Vden);
```

The denominator regularization cfg.v_eps avoids numerical singularity near zero speed. If you see strange low-speed slip spikes, increase v_eps slightly. If you want a sharper slip definition at speed, reduce v_eps after verifying low-speed stability.

10. Changing the tyre force law

The current outer model offers two tyre force options: tan-ellipse and linear-saturated. You select the active option by setting cfg.tyre_model in user_outer_model_options.

```
% In user_outer_model_options:
cfg.tyre_model = 'tan-ellipse';        % combined-slip shape inspired by your older model
% or
cfg.tyre_model = 'linear-saturated';   % simpler fallback model
```

10.1 Replace with your own tyre function

To use your own model, add a new branch in user_tyre_contact_wrench and define a new function. Keep the interface simple: kappa, alpha, Fz, and cfg should produce Fx, Fy, and diagnostic mu_ratio.

```
% In user_tyre_contact_wrench:
if strcmpi(cfg.tyre_model, 'my-tyre')
    [Fx, Fy, mu_ratio] = tyre_force_my_model(cfg, kappa, alpha, Fz);
elseif strcmpi(cfg.tyre_model, 'tan-ellipse')
    [Fx, Fy, mu_ratio] = tyre_force_tan_ellipse(cfg, kappa, alpha, Fz);
else
    [Fx, Fy, mu_ratio] = tyre_force_linear_saturated(cfg, kappa, alpha, Fz);
end

% New local function in the same file:
function [Fx, Fy, mu_ratio] = tyre_force_my_model(cfg, kappa, alpha, Fz)
    Cx = cfg.Ck;
    Cy = cfg.Ca;
    Fx_raw = Cx*kappa;
    Fy_raw = -Cy*alpha;
    limit = cfg.mu*Fz + 1.0e-9;
    scale = min(1.0, limit/sqrt(Fx_raw^2 + Fy_raw^2 + 1.0e-12));
    Fx = Fx_raw*scale;
    Fy = Fy_raw*scale;
```

```
    mu_ratio = sqrt(Fx^2 + Fy^2)/limit;
end
```

Always check the sign of Fy after replacing the tyre law. The current code uses Fy0 = -Ca*alpha in the simple model, and the tan-ellipse model uses a negative tan(alpha) term internally. Match this convention unless you intentionally change the lateral force sign.

11. Road customization: flat road versus ribbon road

Road selection is controlled by the run command and by args.road_type. The flat road is a pure plane with zero elevation and zero bank. The ribbon road uses elevation and bank parameters from the runner options.

Road type	Surface	Use case
flat	z = 0, normal = [0;0;1], longitudinal = [1;0;0], lateral = [0;1;0].	Debugging signs, tyre model, straight-line acceleration, basic steering.
ribbon	Straight 3D road with sinusoidal elevation and banking.	Testing 3D contact, road-normal force direction, suspension response over road profile.

```
% Flat road run:
run_vehicle14_matlab_casadi_demo('simulate','outer','flat')

% Ribbon road run:
run_vehicle14_matlab_casadi_demo('simulate','outer','ribbon')
```

For custom road shapes, edit road_surface_at and centerline_quantities consistently in both the runner and outer model if both contain road helper copies. The road function must provide surface point, longitudinal tangent, lateral tangent, and road normal.

12. Aero customization

Aero is computed in user_aero_model. The default model uses body longitudinal speed, density, drag area, front/rear downforce areas, and ride-height multipliers. Drag inputs are negative in the body/global forward direction convention used by the generated file. Downforce inputs are negative because vertical up is positive.

```
% Turn aero off:
% In user_outer_model_options:
cfg.disable_aero = true;

% Change aero coefficients by adding fields to par or editing defaults:
% CdA, ClA_front, ClA_rear, aero_h_sens_front, aero_h_sens_rear
```

12.1 Example: constant aero for debugging

```
function [FdragF, FdownF, FdragR, FdownR, diag] = user_aero_model(t_abs, x, S, par, args, road, meta, cfg)
    if t_abs < cfg.drop_settle_time
        FdragF = 0; FdragR = 0; FdownF = 0; FdownR = 0;
    else
        FdragF = -50; FdragR = -50;
        FdownF = -300; FdownR = -500;
    end
    diag.enabled = true;
end
```

13. Closed-loop or controller-based custom simulations

You can compute controls from the current state x. For example, x(15) is body-frame longitudinal speed vx and x(20) is yaw rate r. A simple speed controller can compute total drive/brake torque from target speed.

```
% Example inside user_control_signals after tau is defined:
vx = x(15);                % body-frame longitudinal speed
v_target = 20.0;            % m/s
err = v_target - vx;

Kp_drive = 120.0;           % N m per (m/s)
Kp_brake = 250.0;           % N m per (m/s)
Tt_direct = max(Kp_drive*err, 0.0);
Tb_direct = max(-Kp_brake*err, 0.0);

% Simple steering sine input for path disturbance testing:
delta = deg2rad(2.0)*sin(2*pi*0.25*tau);

ctrl.T_drive_total = min(Tt_direct, cfg.T_drive_max);
ctrl.T_brake_total = min(Tb_direct, cfg.T_brake_max);
ctrl.delta = delta;
```

Do not make the outer model depend on xdot. The solver can evaluate the residual many times while searching for xdot, so keeping the outer model algebraic in t and x is much more robust.

14. Recommended debugging workflow

When you create a new custom input signal or tyre model, use a staged debug process. This reduces the chance of mixing a control bug with a contact or road bug.

Stage	Command/setup	What to check
1	run_vehicle14_matlab_casadi_demo('simulate','outer','flat') with mild controls	Vehicle settles, no huge normal load spikes, no unexpected yaw/lateral motion.
2	Straight launch on flat road	vx increases, rear wheel torque positive, braking torque zero.
3	Braking test on flat road	Wheel torques become negative during braking, vx decreases.
4	Small steering on flat road	yaw rate and lateral response have sensible sign and magnitude.
5	Same controls on ribbon road	Suspension travel and normal loads respond to elevation/banking.
6	Aggressive controls only after stages 1-5 pass	No solver blow-up, no unrealistic compression/loads.

The plots to inspect first are driver commands, vehicle response channels, tyre compression, wheel normal loads, and 3D trajectory. Large negative compression after contact, normal loads collapsing to zero while driving, or opposite-sign yaw response usually indicate a sign or geometry problem.

15. Checklist before trusting a custom simulation

- The run command explicitly uses outer, for example run_vehicle14_matlab_casadi_demo('simulate','outer','flat').
- delta and delta_dot are consistent. If you changed steering, you also changed steering_only or implemented a consistent derivative.
- T_drive_total is nonnegative for drive and T_brake_total is nonnegative for brake magnitude.
- Wheel torque signs are correct: positive drive, negative brake.
- The tyre force model returns Fx and Fy with the same sign convention as the rest of the file.
- The tyre model uses Fz >= 0 and handles Fz close to zero safely.
- The outer model depends on t, x, S, parameters, and road, but not directly on xdot.
- Flat-road tests pass before running the 3D road.
- The output file name includes outer/original and flat/ribbon, so old results are not being confused with new results.

16. Quick reference: most common edits

Goal	Edit this function/setting	Minimal change
Change maneuver timing	user_outer_model_options	Change throttle_start, steer_start, brake_start, ramps, maneuver_time.
Change torque levels	user_outer_model_options	Change T_drive_max or T_brake_max.
Use direct torque commands	user_control_signals	Assign ctrl.T_drive_total and ctrl.T_brake_total directly.
Use measured signals	user_control_signals and steering_only	Use interp1 for Tt, Tb, delta; keep delta_dot consistent.
Change brake bias	user_outer_model_options	Change cfg.brake_bias_front.
Disable left/right brake redistribution	user_outer_model_options	Set cfg.enable_lateral_brake_redistribution = false.
Change tyre law	user_outer_model_options and tyre force functions	Set cfg.tyre_model and add/modify tyre_force_* function.
Disable aero	user_outer_model_options	Set cfg.disable_aero = true.
Run flat road	MATLAB command	run_vehicle14_matlab_casadi_demo('simulate','outer','flat')

17. Minimal custom simulation template

The following is a compact template for a custom flat-road test using the outer model. It starts with a gentle launch, adds a single steering pulse, and then brakes.

```
% 1) In user_outer_model_options:
cfg.control_profile = 'my_custom_test';
```

```
cfg.T_drive_max = 1900.0;
cfg.T_brake_max = 4500.0;
cfg.tyre_model = 'linear-saturated';
cfg.disable_aero = true;   % optional for first debug

% 2) In user_control_signals, add:
elseif strcmp(profile, 'my_custom_test')
    throttle = 0.35*smooth_step(tau, 0.2, 1.2) * (1.0 - smooth_step(tau, 8.0, 8.5));
    brake = 0.25*smooth_step(tau, 9.0, 9.5) * (1.0 - smooth_step(tau, 11.0, 11.5));
    steer_on = smooth_step(tau, 4.0, 4.5);
    steer_off = smooth_step(tau, 6.0, 6.5);
    delta = deg2rad(3.0)*steer_on*(1.0 - steer_off);

% 3) In steering_only, add matching steering only:
elseif strcmp(profile, 'my_custom_test')
    steer_on = smooth_step(tau, 4.0, 4.5);
    steer_off = smooth_step(tau, 6.0, 6.5);
    delta = deg2rad(3.0)*steer_on*(1.0 - steer_off);

% 4) Run:
run_vehicle14_matlab_casadi_demo('simulate','outer','flat')
```

18. Final advice

Start by changing only one thing at a time. First use the existing tyre model with new control signals. Then test a new tyre law on simple flat-road cases. Then add aero and 3D road effects. This staged process makes it much easier to identify whether a problem comes from controls, torque split, tyre forces, road geometry, or solver settings.

The outer model is the correct customization point because it is designed to convert your controls and physical submodels into the generated input vector while leaving the generated dynamics unchanged.